

Week 2 Assignment — Solutions & Code Documentation

Course: STEPCLASSES | Topic: Strings, Methods, ASCII, StringBuilder & Best Practices | Branch: WEEK2

Submission Guidelines & Best Programming Practices Followed:

- **Variables:** Descriptive naming distinguishing constants, inputs, and results.
- **Modular Methods:** All core logic implemented in reusable methods; main() handles I/O only.
- **Naming Conventions:** Proper camelCase for methods/variables, PascalCase for classes.
- **Exception Handling:** Try-catch-finally blocks with proper resource cleanup for scanners.
- **Performance & Strings:** Used StringBuilder for mutable operations, avoided regex loops.

Problem 1: ATM PIN Length Validator

Scenario: Checks that a customer PIN is exactly 4 digits long before allowing continuation.
Signature: void checkPinLength(String pin)
Concepts: length(), if / else, comparison operators, exception safety.

Sample Input	Expected / Actual Execution Output
"482"	Invalid PIN - must be exactly 4 digits.
"4820"	PIN length OK.

Source Code (PROGRAM1.java):

```
import java.util.Scanner;

/**
 * Problem 1: ATM PIN Length Validator
 * Validates that an entered PIN string is exactly 4 digits long.
 */
public class PROGRAM1 {

    /**
     * Checks whether the PIN length is exactly 4.
     *
     * @param pin the entered PIN string
     */
    public static void checkPinLength(String pin) {
        if (pin == null) {
            System.out.println("Invalid PIN - must be exactly 4 digits.");
            return;
        }

        int pinLength = pin.length();
        if (pinLength != 4) {
            System.out.println("Invalid PIN - must be exactly 4 digits.");
        } else {
            System.out.println("PIN length OK.");
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.print("Enter PIN: ");
            if (scanner.hasNextLine()) {
                String inputPin = scanner.nextLine();
                checkPinLength(inputPin);
            }
        } catch (Exception e) {
            System.err.println("An error occurred while reading input: " + e.getMessage());
        }
    }
}
```

```
    } finally {  
        scanner.close();  
    }  
}
```

Problem 2: Word Reversal Encoder

Scenario: Reverses every word in a sentence individually while keeping word order the same.

Signature: String reverseEachWord(String sentence)

Concepts: split(), loop reversal, StringBuilder, string joining.

Sample Input	Expected / Actual Execution Output
"hello club"	olleh bulc

Source Code (PROGRAM2.java):

```
import java.util.Scanner;

/**
 * Problem 2: Word Reversal Encoder
 * Reverses each word individually in a sentence while maintaining word order.
 */
public class PROGRAM2 {

    /**
     * Reverses each word individually within the sentence.
     *
     * @param sentence the input sentence
     * @return the sentence with each individual word reversed
     */
    public static String reverseEachWord(String sentence) {
        if (sentence == null || sentence.isEmpty()) {
            return "";
        }

        String[] words = sentence.split(" ");
        StringBuilder reversedSentence = new StringBuilder();

        for (int i = 0; i < words.length; i++) {
            String word = words[i];
            StringBuilder reversedWord = new StringBuilder();

            // Reverse the current word using a loop
            for (int j = word.length() - 1; j >= 0; j--) {
                reversedWord.append(word.charAt(j));
            }

            reversedSentence.append(reversedWord);

            // Append space between words if not the last word
            if (i < words.length - 1) {
                reversedSentence.append(" ");
            }
        }

        return reversedSentence.toString();
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.print("Enter sentence: ");
            if (scanner.hasNextLine()) {
                String inputSentence = scanner.nextLine();
                String result = reverseEachWord(inputSentence);
                System.out.println(result);
            }
        } catch (Exception e) {
            System.err.println("An error occurred while processing input: " + e.getMessage());
        } finally {
            scanner.close();
        }
    }
}
```

```
}  
}  
}
```

Problem 3: Product Inventory CSV Parser

Scenario: Splits CSV lines ("ProductName,SKU,Quantity") into fields and prints a formatted record.

Signature: void parseInventoryRecord(String csvLine)

Concepts: split(", "), array length validation, trimming, string concatenation.

Sample Input	Expected / Actual Execution Output
"Wireless Mouse,WM-2201,150"	Product: Wireless Mouse SKU: WM-2201 Qty: 150
"Wireless Mouse,150"	Invalid Record

Source Code (PROGRAM3.java):

```
import java.util.Scanner;

/**
 * Problem 3: Product Inventory CSV Parser
 * Parses a CSV line in the format "ProductName,SKU,Quantity" and displays a formatted inventory record.
 */
public class PROGRAM3 {

    /**
     * Parses and formats an inventory record from a CSV line.
     *
     * @param csvLine comma-separated input string
     */
    public static void parseInventoryRecord(String csvLine) {
        if (csvLine == null || csvLine.trim().isEmpty()) {
            System.out.println("Invalid Record");
            return;
        }

        String[] fields = csvLine.split(",");

        // Validate that exactly 3 fields are present
        if (fields.length != 3) {
            System.out.println("Invalid Record");
            return;
        }

        String productName = fields[0].trim();
        String sku = fields[1].trim();
        String quantity = fields[2].trim();

        System.out.println("Product: " + productName + " | SKU: " + sku + " | Qty: " + quantity);
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.print("Enter CSV line: ");
            if (scanner.hasNextLine()) {
                String inputLine = scanner.nextLine();
                parseInventoryRecord(inputLine);
            }
        } catch (Exception e) {
            System.err.println("An error occurred while parsing record: " + e.getMessage());
        } finally {
            scanner.close();
        }
    }
}
```

Problem 4: Library ISBN Normalizer & Validator

Scenario: Normalizes 13-character ISBNs (3 letters, 4 year digits, 6 catalog digits) and validates format.

Signature: String normalizeCode(String raw) + String validateAndFormat(String code)

Concepts: trim(), substring(), Character.isLetter(), Character.isDigit(), StringBuilder.

Sample Input	Expected / Actual Execution Output
" pen2026004251 "	[PEN] YEAR: 2026 CATALOG: 004251
"12N2026004251"	Invalid: publisher code must be 3 letters

Source Code (PROGRAM4.java):

```
import java.util.Scanner;

/**
 * Problem 4: Library ISBN Normalizer & Validator
 * Normalizes and validates 13-character ISBN-style codes:
 * 3 letters (publisher code) + 4 digits (year) + 6 digits (catalog number).
 */
public class PROGRAM4 {

    /**
     * Normalizes raw code: trims whitespace and uppercases only the first 3 characters.
     *
     * @param raw the input raw code string
     * @return normalized string
     */
    public static String normalizeCode(String raw) {
        if (raw == null) {
            return "";
        }
        String trimmed = raw.trim();
        if (trimmed.length() < 3) {
            return trimmed.toUpperCase();
        }
        // Uppercase only the first 3 characters using substring() + concatenation
        return trimmed.substring(0, 3).toUpperCase() + trimmed.substring(3);
    }

    /**
     * Validates normalized code and returns formatted line or error reason.
     *
     * @param code the normalized code string
     * @return formatted record string or validation error message
     */
    public static String validateAndFormat(String code) {
        if (code == null || code.length() != 13) {
            return "Invalid: wrong length";
        }

        // Validate first 3 characters are letters without regex
        for (int i = 0; i < 3; i++) {
            if (!Character.isLetter(code.charAt(i))) {
                return "Invalid: publisher code must be 3 letters";
            }
        }

        // Validate remaining 10 characters are digits without regex
        for (int i = 3; i < 13; i++) {
            if (!Character.isDigit(code.charAt(i))) {
                return "Invalid: non-digit body";
            }
        }

        String pubCode = code.substring(0, 3);
```

```
String year = code.substring(3, 7);
String catalogNumber = code.substring(7);

StringBuilder sb = new StringBuilder();
sb.append("[").append(pubCode).append("] YEAR: ").append(year)
  .append(" | CATALOG: ").append(catalogNumber);

return sb.toString();
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    try {
        System.out.print("Enter ISBN code: ");
        if (scanner.hasNextLine()) {
            String rawInput = scanner.nextLine();
            String normalized = normalizeCode(rawInput);
            String result = validateAndFormat(normalized);
            System.out.println(result);
        }
    } catch (Exception e) {
        System.err.println("An error occurred during validation: " + e.getMessage());
    } finally {
        scanner.close();
    }
}
```

Problem 5: Stop-Word-Filtered Word Frequency Report

Scenario: Excludes stop words ('the','was','and','a','is','of','in'), counts word frequencies, and sorts descending.

Signature: void printFilteredWordFrequency(String feedback)

Concepts: replace() cleaning, split("\\s+"), Set lookup, HashMap counting, comparator sort.

Sample Input	Expected / Actual Execution Output
"The mentor was great, the session was great and clear."	great: 2 mentor: 1 session: 1 clear: 1

Source Code (PROGRAM5.java):

```
import java.util.*;

/**
 * Problem 5: Stop-Word-Filtered Word Frequency Report
 * Performs frequency analysis of feedback paragraphs excluding filler words.
 */
public class PROGRAM5 {

    // Predefined stop-words list
    private static final Set<String> STOP_WORDS = new HashSet<>(
        Arrays.asList("the", "was", "and", "a", "is", "of", "in")
    );

    /**
     * Normalizes text, filters stop words, counts word frequencies,
     * and prints unique words sorted in descending order of count.
     *
     * @param feedback paragraph text
     */
    public static void printFilteredWordFrequency(String feedback) {
        if (feedback == null || feedback.trim().isEmpty()) {
            return;
        }

        // Normalize: convert to lowercase and strip punctuation (periods and commas) using replace()
        String cleaned = feedback.toLowerCase()
            .replace(".", "")
            .replace(",", "")
            .replace("!", "")
            .replace("?", "")
            .replace(";", "")
            .replace(":", "");

        // Split into words using whitespace pattern
        String[] words = cleaned.trim().split("\\s+");

        // Count frequency of non-stop-words
        Map<String, Integer> frequencyMap = new HashMap<>();
        for (String word : words) {
            if (word.isEmpty() || STOP_WORDS.contains(word)) {
                continue;
            }
            frequencyMap.put(word, frequencyMap.getOrDefault(word, 0) + 1);
        }

        // Sort entries by count in descending order
        List<Map.Entry<String, Integer>> sortedList = new ArrayList<>(frequencyMap.entrySet());
        sortedList.sort((entry1, entry2) -> entry2.getValue().compareTo(entry1.getValue()));

        // Print results
        for (Map.Entry<String, Integer> entry : sortedList) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }
    }
}
```

```
    }  
}  
  
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    try {  
        System.out.println("Enter feedback text (type an empty line to finish:");  
        StringBuilder feedbackBuilder = new StringBuilder();  
        while (scanner.hasNextLine()) {  
            String line = scanner.nextLine();  
            if (line.isEmpty()) {  
                break;  
            }  
            feedbackBuilder.append(line).append(" ");  
        }  
        printFilteredWordFrequency(feedbackBuilder.toString());  
    } catch (Exception e) {  
        System.err.println("An error occurred: " + e.getMessage());  
    } finally {  
        scanner.close();  
    }  
}
```